IT CAREER HUB

Django

Additional practicum: ORM queries, Serializers & basic views



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- Камера должна быть включена на протяжении всего занятия
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Вести себя уважительно и этично по отношению к остальным участникам занятия
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

IT CAREER HUB

ПРАКТИЧЕСКАЯ РАБОТА

Задание 1

Проекты за текущий месяц

1. Импортируйте модель Project.
2. Импортируйте модуль timezone из django.
3. Напишите запрос, который выдаст только те проекты, которые были созданы в текущем месяце (месяц брать от даты текущего дня).
4. Если есть хоть один проект - выведите информацию:
    - Название проекта
    - Полная дата его создания

Задание 1

Решение

```
from django.utils import timezone
from management_app.models.project import Project
cur_month_projects = Project.objects.filter(date_of_creation__month=timezone.now().month)

if cur_month_projects:
   for proj in cur_month_projects:
       print("=" * 50)
       print(proj.name)
       print(proj.date_of_creation)
else:
   print("EMPTY DATA")
```

Задание 2

Файлы проектов в день недели

1. Импортируйте модель ProjectFile.
2. Импортируйте класс ExtractWeekDay из django.
3. Напишите запрос, который будет получать список файлов за определённый день, например понедельник:
    - Для выборки конкретного значения по полю даты создания используйте класс ExtractWeekDay.
    - Для упрощения работы с таким значением задайте для него новое, не заявленное в модели поле, например weekday.
    - Проведите фильтрацию по этому новому полю.
4. Если файлов не было найдено - выведите сообщение “Empty Data”, если был найден хоть один файл - выведите информацию о нём:
    - Название файла
    - Путь к файлу, где он сохранён

Задание 2

Решение

```
from django.db.models.functions import ExtractWeekDay
from management_app.models.project import ProjectFile

monday_project_files = ProjectFile.objects.annotate(
   weekday=ExtractWeekDay('created_at')
).filter(
   weekday=2
)

if monday_project_files:
   for file_ in monday_project_files:
       print("=" * 50)
       print(file_.file_name)
       print(file_.file_path)
else:
   print("EMPTY DATA")
```

Задание 3

Общее количество проектов

1. Импортируйте модель Project.
2. Напишите запрос, который будет считать количество всех проектов, которые есть в базе данных.
3. Выведите количество всех проектов в консоль.

Задание 3

Решение

```
from management_app.models.project import Project

count_of_projects = Project.objects.all().count()

print(f"Count of projects: {count_of_projects}")
```

Задание 4

Количество файлов для каждого проекта

1. Импортируйте модель Project.
2. Импортируйте класс Count из django.
3. Напишите запрос, который будет считать количество файлов для каждого проекта персонально.
4. Выведите информацию о количестве файлов для каждого проекта.

Задание 4

Решение

```
from django.db.models import Count
from management_app.models import Project

projects = Project.objects.all()

project_file_counts = {}

for project in projects:
   file_count = project.project_files.aggregate(num_files=Count('id'))
   project_file_counts[project.name] = file_count['num_files']

for project_name, num_files in project_file_counts.items():
   print(f"Project: {project_name}, Number of files: {num_files}")
```

Задание 5

Среднее количество задач на каждом проекте

1. Импортируйте модель Project.
2. Импортируйте класс AVG из django.
3. Напишите запрос, который сможет для каждого проекта выведите среднее количество задач.
4. Выведите полученные данные.

Задание 5

Решение

```
from django.db.models import Avg
from management_app.models import Project
average_project_tasks = Project.objects.annotate(avg_tasks=Avg('tasks__id'))

for project in average_project_tasks:
   print(f"Project: {project.name}, Average Number of Tasks: {project.avg_tasks}")
```

IT CAREER HUB

ВОПРОСЫ

Задание 6

Количество задач для каждого пользователя

1. Импортируйте модель User из django.
2. Импортируйте класс Count из django.
3. Напишите запрос, который получит из базы данных количество задач для каждого пользователя.
4. В запросе укажите, что хотим работать только с конкретными полями:
    - Username
    - Наше собственно созданное поле count_of_tasks
5. Выведите как результат никнейм разработчика и количество прикреплённых за ним задач.

Задание 6

Решение

```
from django.db.models import Count
from django.contrib.auth.models import User

count_of_tasks_to_user =
User.objects.annotate(count_of_tasks=Count('tasks__id')).values_list('username',
'count_of_tasks')

print(count_of_tasks_to_user.query)

for username, count_of_tasks in count_of_tasks_to_user:
   print(f"Developer - {username}  |  Count of tasks: {count_of_tasks}")
```

Задание 7

Сортировка задач по приоритету и дате выполнения

1. Импортируйте модель Task.
2. Напишите запрос, который сможет отсортировать все задачи по нескольким полям:
    - Приоритет задачи
    - Дата к которой задача должна быть выполнена
3. Выведите название задачи, приоритет и дату завершения.

Задание 7

Решение

```
from management_app.models import Task


all_tasks = Task.objects.order_by(
   'priority',
   'due_date'
).values_list(
   'name', 'priority', 'due_date'
)

for name, priority, due_date in all_tasks:
   print(f"Task name - {name}  |  Priority: {priority}  |  Due date - {due_date}")
```

Задание 8

Сортировка пользователей по количеству задач

1. Импортируйте модель User из django.
2. Напишите запрос, который будет сортировать всех пользователей исходя из того, как много у каждого задач.
3. Отсортируйте от самого загруженного сотрудника, к самому свободному по задачам.
4. Конкретизируйте запрос таким образом, чтобы в него включались только поля:
    - Username
    - Count_of_tasks
5. Выведите все данные в консоль.

Задание 8

Решение

```
from django.db.models import Count
from django.contrib.auth.models import User

sorted_users = User.objects.annotate(
   count_of_tasks=Count('tasks__id')
).order_by('-count_of_tasks').values_list('username', 'count_of_tasks')

print(sorted_users.query)

for username, count_of_tasks in sorted_users:
   print(f"{username}  |  {count_of_tasks}")
```

Задание 9

Пагинация для задач

1. Импортируйте модель Task.
2. Импортируйте класс Paginator.
3. Напишите запрос для получения всех задач.
4. Реализуйте пагинацию для того, чтобы в рамках одной страницы отображалось ровно 10 задач, не более.
5. Выведите данные об этих 10-ти задачах:
    - Название
    - Статус
    - Приоритетность
    - Username разработчика

Задание 9

Решение

```
from management_app.models import Task
from django.core.paginator import Paginator

tasks = Task.objects.all()

pages_paginator = Paginator(tasks, 10)

page_number = 1

page_tasks = pages_paginator.get_page(page_number)

for task in page_tasks:
   print("=" * 50)
   print(task.name)
   print(task.status)
   print(task.priority)
   print(task.assignee.username)
```

Задание 10

Первый сериализатор для проектов

1. Установите модуль djangorestframework.
2. Добавить этот модуль в файл requirements.txt.
3. Добавить этот модуль в список установленных приложений в файле settings.py.
4. Создать новый пакет serializers, в котором создать первый файл - projects
5. Импортируйте класс ModelSerializer.
6. Напишите первый сериализатор, который будет обрабатывать поля id и name для проекта, выдавая общую базу для вывода всех проектов.

Задание 10

Решение

```
# management_app/serializers/projects
from rest_framework.serializers import ModelSerializer

from management_app.models import Project


class AllProjectsSerializer(ModelSerializer):
   class Meta:
       model = Project
       fields = ['id', 'name']
```

IT CAREER HUB

ВОПРОСЫ

Задание 11

Первое представление для отображения списка всех проектов

1. Импортируйте ранее написанный сериализатор в файл, где будет реализовываться первое представление.
2. Импортируйте дополнительные методы и классы:
    - Request из rest_framework (для типизации объекта запроса)
    - JsonResponse из django (для возврата и типизации ответов)
    - Status из rest_framework (для указания статусов ответов)
    - Api_view из rest_framework.decorators (для конкретики обрабатываемых методов)
    - Модель Project (для запросов к таблице проектов в базе данных)
3. Напишите функцию, которая будет обрабатывать GET запросы для получения списка всех проектов из Базы данных.
4. Зарегистрировать эту функцию в списке эндпоинтов в файле urls.py
5. Проверьте работу готового эндпоинта.

Задание 11

Решение

```
from rest_framework.request import Request
from django.http import JsonResponse
from rest_framework import status
from rest_framework.decorators import api_view

from management_app.serializers.projects import AllProjectsSerializer
from management_app.models.project import Project


@api_view(['GET',])
def get_all_projects(request: Request) -> JsonResponse:
   all_projects = Project.objects.all()

   if not all_projects.exists():
       return JsonResponse(
           [],
           status=status.HTTP_204_NO_CONTENT,
           safe=False
       )
   else:
       serialized_data = AllProjectsSerializer(
           all_projects, many=True
       )
```

1

```
       return JsonResponse(
           serialized_data.data,
           status=status.HTTP_200_OK,
           safe=False
       )

# -----------------------------
# management_app.urls

from django.urls import path
from management_app.views import get_all_projects

urlpatterns = [
   path('projects/', get_all_projects),
]
# -----------------------------
# project_management.urls

from django.urls import path, include

urlpatterns = [
   path('management_app/', include('management_app.urls')),
]
```

2

Задание 12

Сериализатор для модели задач (Task)

1. Создайте новый файл для сериализаторов задач.
2. Импортируйте класс ModelSerializer.
3. Напишите сериализатор для модели Task с полями id, name, status, priority для получения общей информации о задачах.

Задание 12

Решение

```
from rest_framework.serializers import ModelSerializer

from management_app.models import Task


class AllTasksSerializer(ModelSerializer):
   class Meta:
       model = Task
       fields = ['id', 'name', 'status', 'priority']
```

Задание 13

Отображение для получения списка всех задач

1. Импортируйте модель Task
2. Импортируйте написанный ранее сериализатор AllTasksSerializer
3. Напишите отображение, которое будет выводить JSON данные о всех задачах из базы данных:
    - Если при отправке запроса не передавать конкретный проект - выводить все задачи
    - Если был указан конкретный проект, например “TIGER” - выводить только те задачи, которые принадлежат этому проекту

Задание 13

Решение

```
@api_view(['GET',])
def get_all_tasks(request: Request) -> JsonResponse:
   project_name = request.query_params.get('project')

   if not project_name:
       all_tasks = Task.objects.all()

   else:
       all_tasks =
Task.objects.filter(project__name=project_name)

   if not all_tasks.exists():
       return JsonResponse(
           data=[],
           status=status.HTTP_204_NO_CONTENT,
           safe=False
       )

   serialized_data = AllTasksSerializer(all_tasks,
many=True)
```

1

```
   return JsonResponse(
       data=serialized_data.data,
       status=status.HTTP_200_OK,
       safe=False
   )

# -----------------------------
# management_app/urls.py
from management_app.views import get_all_projects,
get_all_tasks


urlpatterns = [
   path('projects/', get_all_projects),
   path('tasks/', get_all_tasks),  # NEW
]
```

2

Задание 14

Сериализатор на получение всех тегов

1. Создайте новый файл tags для хранение сериализаторов.
2. Импортируйте класс ModelSerializer.
3. Напишите сериализатор для получения всех тегов.

Задание 14

Решение

```
from rest_framework import serializers

from management_app.models import Tag


class TagsSerializer(serializers.ModelSerializer):
   class Meta:
       model = Tag
       fields = '__all__'
```

Задание 15

Отображение для получения всех тегов

1. Импортируйте модель Tag.
2. Импортируйте сериализатор AllTagsSerializer.
3. Напишите функцию, которая будет отображать список всех тегов в формате JSON.

Задание 15

Решение

```
@api_view(['GET',])
def get_all_tags(request: Request) -> JsonResponse:
   tags = Tag.objects.all()

   if not tags.exists():
       return JsonResponse(
           data=[],
           status=status.HTTP_204_NO_CONTENT,
           safe=False
       )

   serialized_data = TagsSerializer(tags, many=True)

   return JsonResponse(
       data=serialized_data.data,
       status=status.HTTP_200_OK,
       safe=False
   )
```

1

```
# ----------------------------
# management_app/urls.py
from management_app.views import
get_all_projects, get_all_tasks, get_all_tags

urlpatterns = [
   path('projects/', get_all_projects),
   path('tasks/', get_all_tasks),
   path('tags/', get_all_tags),  # NEW
]
```

2

IT CAREER HUB

ВОПРОСЫ

Задание 16

Отображение для получения тега по id

1. Напишите отображение с принимаемым аргументом tag_id.
2. Проверьте существование такого тега и если он есть - вернуть информацию о нём.

Задание 16

Решение

```
@api_view(['GET',])
def get_tag_by_id(request: Request, tag_id: int) -> JsonResponse:
   try:
       tag = Tag.objects.get(id=tag_id)

   except Tag.DoesNotExist:
       return JsonResponse(
           data={},
           status=status.HTTP_204_NO_CONTENT
       )

   serialized_data = TagsSerializer(tag)


   return JsonResponse(
       data=serialized_data.data,
       status=status.HTTP_200_OK,
   )
```

1

```
# -----------------------------
# management_app/urls.py
from management_app.views import get_all_projects,
get_all_tasks, get_all_tags, get_tag_by_id

urlpatterns = [
   path('projects/', get_all_projects),
   path('tasks/', get_all_tasks),
   path('tags/', get_all_tags),
   path('tags/<int:tag_id>/', get_tag_by_id),  # NEW
]
```

2

Задание 17

Отображение для обновления информации о теге

1. Поставьте в декоратор api_view() метод для обновления ‘PUT’.
2. Напишите запрос, который будет получать конкретный тег по id:
    - Реализуйте проверку на получение объекта тега. Если объекта нет - выведите соответствующее сообщение.
3. Напишите отображение, которое будет принимать из тела запроса новые данные:
    - Реализуйте проверку валидации полученных данных.
4. Обновите полученный объект тега новыми данными и сохраните его.

Задание 17

Решение

```
@api_view(['PUT',])
def update_required_tag(request: Request, tag_id: int) ->
JsonResponse:
   try:
       tag = Tag.objects.get(id=tag_id)
   except Tag.DoesNotExist:
       return JsonResponse(
           {},
           status=status.HTTP_204_NO_CONTENT
       )

   validated_data = TagsSerializer(tag, request.data)

   if validated_data.is_valid(raise_exception=True):
       validated_data.save()

       return JsonResponse(
           validated_data.data,
           status=status.HTTP_200_OK,
       )
```

1

```
# -----------------------------
# management_app/urls.py
from management_app.views import get_all_projects,
get_all_tasks, get_all_tags, get_tag_by_id,
update_required_tag

urlpatterns = [
   path('projects/', get_all_projects),
   path('tasks/', get_all_tasks),
   path('tags/', get_all_tags),
   path('tags/<int:tag_id>/', get_tag_by_id),
   path('tags/<int:tag_id>/update/',
update_required_tag),  # NEW
]
```

2

Задание 18

Представление для создания тега

1. Поставьте в декоратор api_view() метод для обновления ‘POST’.
2. Напишите отображение, которое будет создавать новый объект тега.
    - Реализуйте проверку валидации полученных данных.
3. Сохраните новый объект в базе данных.

Задание 18

Решение

```
@api_view(['POST',])
def create_new_tag(request: Request) -> JsonResponse:
   new_tag = TagsSerializer(data=request.data)

   if new_tag.is_valid(raise_exception=True):
       new_tag.save()

       return JsonResponse(
           new_tag.data,
           status=status.HTTP_201_CREATED
       )
   return JsonResponse(
       new_tag.errors,
       status=status.HTTP_400_BAD_REQUEST
   )
```

1

```
# ----------------------------
# management_app/urls.py
from management_app.views import
get_all_projects, get_all_tasks, get_all_tags,
get_tag_by_id, update_required_tag,
update_required_tag, create_new_tag

urlpatterns = [
   path('projects/', get_all_projects),
   path('tasks/', get_all_tasks),
   path('tags/', get_all_tags),
   path('tags/create/', create_new_tag),  # NEW
   path('tags/<int:tag_id>/', get_tag_by_id),
   path('tags/<int:tag_id>/update/',
update_required_tag),
]
```

2

Задание 19

Представление для удаления тега

1. Поставьте в декоратор api_view() метод для обновления ‘DELETE’.
2. Напишите запрос, который будет получать конкретный тег по id.
3. Напишите отображение, которое будет удалять полученный объект тега и выводить сообщение об успешном удалении.

Задание 19

Решение

```
@api_view(['DELETE',])
def delete_required_tag(request: Request, tag_id: int) ->
JsonResponse:
   try:
       tag = Tag.objects.get(id=tag_id)
   except Tag.DoesNotExist:
       return JsonResponse(
           {},
           status=status.HTTP_204_NO_CONTENT
       )

   tag.delete()
   return JsonResponse(
       {
           'message': 'Tag deleted successfully',
       },
       status=status.HTTP_200_OK
   )
```

1

```
# -----------------------------
# management_app/urls.py
from management_app.views import get_all_projects,
get_all_tasks, get_all_tags, get_tag_by_id,
update_required_tag, update_required_tag,
create_new_tag, delete_required_tag

urlpatterns = [
   path('projects/', get_all_projects),
   path('tasks/', get_all_tasks),
   path('tags/', get_all_tags),
   path('tags/create/', create_new_tag),  # NEW
   path('tags/<int:tag_id>/', get_tag_by_id),
   path('tags/<int:tag_id>/update/',
update_required_tag),
   path('tags/<int:tag_id>/delete/',
delete_required_tag),
]
```

2

Задание 20

Подробная информация о задаче

1. Напишите сериализатор TaskInfoSerializer для получения подробной информации о задаче.
2. Поле для тегов должно быть вложенным объектом, которым является уже готовый сериализатор TagsSerializer.

Задание 20

Решение

```
from management_app.serializers.tags import TagsSerializer


class TaskInfoSerializer(serializers.ModelSerializer):
   tags = TagsSerializer(many=True)

   class Meta:
       model = Task
       fields = [
           'id',
           'name',
           'status',
           'priority',
           'tags',
           'project',
           'created_date',
           'due_date'
       ]
```

Задание 21

Детальная информация о задаче

1. Импортируйте сериализатор TaskInfoSerializer.
2. Напишите функцию с аргументом task_id для получения детальной информации о задаче:
    - Проверьте существование задачи по её id. Если задачи нет - выведите соответствующее сообщение, если задача есть - выведите её содержимое.

Задание 21

Решение

```
@api_view(['GET',])
def get_task_by_id(request: Request, task_id: int) ->
JsonResponse:
   try:
       task = Task.objects.get(id=task_id)
   except Task.DoesNotExist:
       return JsonResponse(
           {},
           status=status.HTTP_204_NO_CONTENT
       )


   serializer_data = TaskInfoSerializer(task)


   return JsonResponse(
       serializer_data.data,
       status=status.HTTP_200_OK,
   )
```

1

```
# –----------------------------
# management_app/urls.py
from management_app.views import get_all_projects,
get_all_tasks, get_all_tags, get_tag_by_id,
update_required_tag, update_required_tag,
create_new_tag, delete_required_tag, get_task_by_id

urlpatterns = [
   path('projects/', get_all_projects),
   path('tasks/', get_all_tasks),
   path('tasks/<int:task_id>/', get_task_by_id),  # NEW
   path('tags/', get_all_tags),
   path('tags/create/', create_new_tag),
   path('tags/<int:tag_id>/', get_tag_by_id),
   path('tags/<int:tag_id>/update/',
update_required_tag),
   path('tags/<int:tag_id>/delete/',
delete_required_tag),
]
```

2

IT CAREER HUB

ВОПРОСЫ

Заключение

